WorldPath - A series of points that track the movement of an agent through a world. These should rapidly be able to give the user information about the path, such as its length, whether it crosses back on itself and perhaps additional calculated information such as the furthest two points apart. These will be useful in carrying out an agent's behavior (for example, laying out a path for it to follow.)

1. Class Description: World Path will store and analyze world positions representing an agent's movement history and or a planned route to follow. It will record where the agent has been in the world to detect backtracking as well as store a planned route for the agent to target. Could also store paths for repeated movements.

2. Similar classes in the standard library: std::vector<>, cmath

3. Key functions:

    a. WorldPath()

    b. WorldPath(std::vector<Point> points)

    c. Void clear()

    d. Void push_back(const Point& p)

    e. Double length() const

    f. Bool self_intersect()

    g. Const Point& start()

    h. Const Point& end()

4. Error Conditions:

    a. Programmer error: Checking for any negatives that don't belong

    b. Recoverable error: Need to be mindful of memory allocation

    c. User error: Need assertions to make sure there are no invalid inputs from user

5. Expected Challenges:

    a. Performance tradeoffs

    b. Self-Intersection behavior

6. Other groups' classes:

    a. Group 5 AnnotationSet and TagManager: WorldPath would benefit from agents being able to remember things that they have encountered along their path

    b. Group 7 StateGrid/StateGridPosition: Creates the world that WorldPath would be used on

PathGenerator

1. Class Description

This class will generate a path through the world. This path will be from Point A to Point B, and can be different depending on the type of path that is best needed for the situation.

2. Related / Similar Standard Library Classes
- Std::priority_queue
- Std::vector
- Std::unordered_set
- std::unordered_map

3. Key Functions

- class PathStatus { Ok, NoPath, InvalidStart, InvalidGoal, BudgetExceeded}
- class PathType { Shortest, LowCost, Patrol, Avoid}
- Struct PathResult {WorldPath path; PathStatus status}
- PathGenerator(World nav)
- SetBudget(Int maxNodesExpanded)
- PathResult CreatePath(Point a, Point b, PathType type = PathType::Shortest)
- PathResult CreatePatrolPath(Point a, Point b, std::vector<Point> waypoints)
- PathResult CreateAvoidPath(Point a, Point b, Tag avoid)

4. Error conditions

1. Programmer errors
    - These errors occur when the programmer inputs invalid arguments to these Functions.
    - This will result in assert errors being thown of "invalid arguments"

2. Recoverable errors
    - This error occurs when resource limitation is exceeded
    - This will result in exception occurring, initializing a BudgetExceeded PathStatus

3. User error
    - This error will occur when user inputs invalid data
    - This will result in exception, with PathStatus having either InvalidStart, NoPath, or InvalidGoal.

5. Expected challenges

1. Coordinating the navigation with the world interface
2. Making sure path searches don't exceed budget limit
3. Defining PathTypes that match actual agent goals

6. Class Coordination

1. Group 6: WorldPath
2. Group 6: BehaviorTree
3. Group 7/8 Worlds
4. Group 23 Data Analytics

BehaviorTree

1.)The main goal of BehaviorTree is to provide a structured, rule based logic controller for a classic agent. The class aims to organize decision making by breaking down agent behaviors into a hierarchy of smaller manageable tasks. Enable reusability by creating a library of “classic” logic nodes that can be reused across different agents without rewriting the core code. Maintain state transparency with a clear “trace” of the agent’s logic path so the programmer can see exactly which rule is currently being executed. The high level functionality of the BehaviorTree class is to manage a collection of nodes in a tree structure. It operates on a “Tick” system, where the entire logic tree is evaluated at regular intervals. The execution flow works by the tree being “ticked” then it starts at the root and traverses downward.

2.) Similar Classes in the standard library:

std::expected<T, E> A node’s tick() function needs to return a specific state(Success, Failure, or Running). Std::expected is designed to hold either an expected value or an unexpected error. Using a similar pattern allows the return of a Status of a node.

std::variant<T1, T2, ...> would be for the “Memory Map”. It allows us to store different types of data (integers for health, strings for names, or *`WorldPath`* objects for navigation).

std::map/std::unordered_map is directly related to the “Memory Map”. It's used to store the agent’s knowledge.

std::unique_ptr and **std::shared_ptr**

3.) Key Functions

- **void SetRoot(std::shared_ptr<Node> node)** : Defines starting point of agents logic
- **Void AddChild(CompositeNode& parent, std::shared_ptr<Node> child)** : Controls the building of the tree hierarchy.
- **Status Tick():** Initiates a traversal from the root and returns the resulting state
- **void Reset():** Clears any persisted “Running” states, forcing the logic to restart from the root on the next tick

4.) Error Conditions

- **Null Node Attatchment:** Passing a nullptr into the SetRoot or AddChild functions, (1) programmer error. Caught by an assert(). The tree cannot “tick” a nullptr.
- **Circular Logic:** Programmer accidentally attaches a node as a child of itself, (1) programmer error. Caught by an assert(). Behavior tree must be acyclic.
- **Invalid Type Access:** A node tries to read a memory key as an int, but it was stored as a WorldPath, (3) User/Logic error. Return a special “Error” status.

5.) Challenges

- **Type Safety in the Memory Map:** Using std::variant for the memory map is powerful but managing it safely may be something difficult. We need to ensure nodes don't crash the program when they attempt to retrieve the "wrong" type.
- A primary hurdle is not knowing the company's final product. Because the project scope could range from spatial social NPCs (like The Sims) or text-based productivity chatbots, we face the challenge of designing a BehaviorTree that is neutral.

6.) Coordination with Other Group Classes

- Group 5: AnnotationSet and TagManager: Behavior Tree’s leaf nodes will need to query an agents AnnotationSet to make social decisons
- Group 7 StateGrid and DataMap: StateGrid provides spatial context for our agents

ActionMap

1. Description: ActionMap is a strings to functions registry which allows the user to type in string commands to interact with the project. Its primary function will be supporting the project’s dynamic interface, mapping user thoughts to actions like creating agents, adding objects, and so on. It will contain a simple API that allows programmers to add functions via string/function, and then a trigger function to run what they would like to.

2. Similar classes in STD:
    a. std::unordered_map<std::string, T>: O(1) alternative to map
    b. std::map<std::string, T>: standard map function for string keys to any value
    c. std::invoke: a function that is used for function invocation
    d. std::any: a handler for a variety of argument types with differing signatures
3. Key Functions

a. void AddFunction(std::string name, std::function<void()> func): This function will be responsible for adding to the registry of zero argument functions that the user can invoke.
b. template<typename.. argos> AddFunction(std::string name, std::function<void(args…)> func): registers a function that needs arguments.
c. void Trigger(std::string name): Responsible for looking up a function in the registry and invoking for use by the user.
d. template<typename.. argos> Trigger(std::string name, std::function<void(args…)> func): triggers a function that needs arguments.
e. void Clear(): Removes all functions from the registry
f. bool Remove(std::string name): Removes a specified function from the registry
g. bool HasFunction(std::string name) const: Returns a bool if the specified function exists in the function registry.
h. std::vector<std::string> GetFunctionNames() const: returns a list of registry funcs.

4. Error Conditions:
    a. Adding a function that exists, this can be minimized by using assert(!HasFunction(name)) during development, or creating another ReplaceFunction that overwrites existing functions.
    b. Type mismatch when adding or using functions. Again, assert or other error handling can be used to ensure that the passed function has the expected type and arguments for the called function.

5. Expected challenges and topics to learn
    a. Type erasure with arguments: Supporting functions with different signatures will end up creating multiple maps, so tracking each one will be necessary. I expect to learn how to perfect the forwarding necessary for this.
    b. In hand, function signature validation will be necessary to ensure that the correct function is add/removed/triggered when it is called by the user. We will need to do runtime id/type checking to ensure this.
    c. Performance optimization will be difficult if using an unordered map, as returning a list of all included function signatures will be difficult to ensure.
6. Coordination with other groups:
    a. WebButton (Group 24): This will heavily use ActionMap to trigger and register functions that the user can interact with by using buttons in the game's interface. It may also use GetFunctionNames to dynamically populate menus and buttons with available actions.
    b. EventQueue (Group 8): Storing event names a strings in the overall events of the worlds will certainly use ActionMap to execute the corresponding events when they are processed from the overall queue.

MemoryFactory

1. Class Description: MemoryFactory is a fixed-size pool allocator specialized for one C++ type at a time. It provides fast allocation as well as reclamation for objects that are frequently created and destroyed. This is for things like agents, behavior-tree nodes, path nodes, events, etc.. Instead of calling new/delete for every object the class allocates memory in chunks and maintains a free list of available slots.

2. Similar/informing classes in the standard library:
   - std::allocator<T>: allocator model
   - std::vector<T>: contiguous storage
   - std::unique_ptr<T, Deleter>: allows custom ownership semantics, Enables RAII for non standard memory management

3. Key Functions:
   a. MemoryFactory(): Responsible for initializing internal data structure while performing no object construction itself
   b. ~MemoryFactory(): Responsible for cleaning up all memory owned by the factory.
   c. Template <typename… Args> T* Create(Args&& … args): Allocates a slot, and constructs T
   d. void Destroy(T* ptr): Validates ptr, then call ptr->~T(), returns spot to free list
   e. void Reserve(size_t count): Pre allocate enough blocks for what value count holds
   f. void Clear(): destroy all live objects and reset the free list

4. Error Conditions:
   a. Programmer Error
      i. Destroy(nullptr): use assert(ptr != nullptr
      ii. Use after destroy: We would overwrite freed memory with a pattern. We could also store a debug version number per slot.
   b. Recoverable Error
      i. Out of memory when allocating a new block: In this case allocation methods would return something like std::unexpected(Error::OutOfMemory) value with a message about being out of memory.
      ii. Failing early with Reserve(): This method attempts to allocate enough blocks for count objects up front. Can you use std::expected<void, Error> so it can return {} on success, and std::unexpected(Error:OutOfMemory) on failure.

5. Expected Challenges and Topics to learn
   a. Correct free-list implementation: constant-time allocate/free, learning how to build the data structure in C++
   b. Alignment: making sure that each type is byte aligned properly.
   c. Unit testing: verifying counts, ownerships, failure, etc..

    d. Thread safety: If the allocator has to be thread-safe, could possibly use locks like std::lock_guard or atomics with std::atomic.

6. Coordination with other groups
    a. Group 8 (Interaction-Heavy World): may need a fast allocator for events. Their EventQueue or MemoFunction might benefit.
    b. Group 23 (Analytics): If they want allocation metrics, we would expose counters for them to call and use.

Main Module Write up

Utilizing our behavior tree, world path, path generator, action map, and memory factory classes our classic agent module will be able to act in various social world based behaviors. These behaviors include exploration, decision-making, path analysis, action executables and memory management. Our agent module will use these behaviors and pre-defined logic to interact with the world. These agents can be set to gather or explore or build depending on the world and scenario that the agents are put into.